

# Kubernetes Events and Log Ingestion

---

**Series:** K8S | **Notebook:** 7 of 12 | **Created:** January 2026 | **Last Updated:** 01/30/2026

## Capturing and Analyzing Kubernetes Events and Logs

---

Kubernetes events and container logs provide crucial insights for debugging and operational awareness. This notebook covers event monitoring, log ingestion configuration, and analysis patterns in Dynatrace.

---

## Table of Contents

---

1. [Event Ingestion Configuration](#)
  2. [Container Log Collection](#)
  3. [OpenPipeline for K8s Logs](#)
  4. [Event Analysis Patterns](#)
  5. [Log Analysis Patterns](#)
  6. [Alerting on Events and Logs](#)
-

# Prerequisites

| Requirement           | Details                                                                      |
|-----------------------|------------------------------------------------------------------------------|
| Dynatrace Environment | SaaS with log ingestion enabled                                              |
| DynaKube              | ActiveGate with <code>kubernetes-monitoring</code>                           |
| Permissions           | <code>logs.read</code> , <code>logs.ingest</code> , <code>events.read</code> |
| Knowledge             | K8S-01 Fundamentals                                                          |

## 1. Kubernetes Events Overview

### Event Types

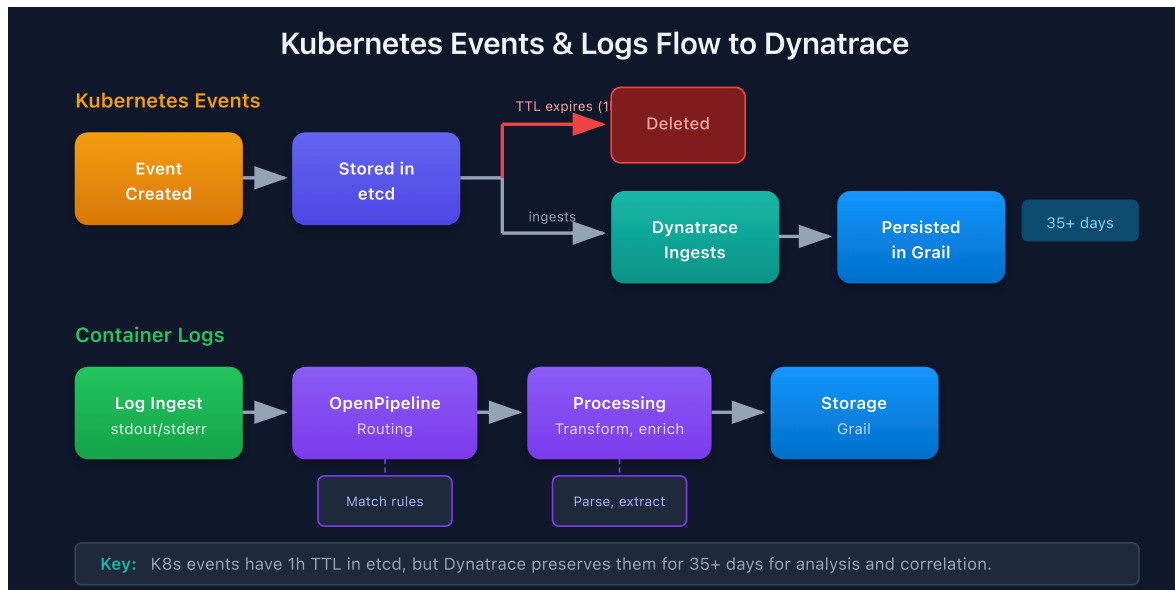
Kubernetes events are first-class objects that record what happened in the cluster.

| Type    | Description        | Examples                             |
|---------|--------------------|--------------------------------------|
| Normal  | Routine operations | Scheduled, Pulled, Created           |
| Warning | Potential issues   | FailedScheduling, BackOff, Unhealthy |

### Event Sources

| Source             | Events Generated                        |
|--------------------|-----------------------------------------|
| Scheduler          | Scheduling decisions, failures          |
| Kubelet            | Pod lifecycle, probe results            |
| Controller Manager | ReplicaSet scaling, Deployment rollouts |
| Custom Controllers | CRD reconciliation events               |

## Event Lifecycle



**Important:** Kubernetes events are short-lived. Dynatrace captures them for long-term storage and analysis.

## 2. Event Ingestion Configuration

### ActiveGate Kubernetes Monitoring

Enable event collection via DynaKube:

```
spec:
  activeGate:
    capabilities:
      - kubernetes-monitoring # Enables event collection
      - routing
```

### Event Filtering

Configure which events to ingest:

| Setting          | Location                            | Purpose                          |
|------------------|-------------------------------------|----------------------------------|
| Event types      | Settings > Cloud and virtualization | Normal, Warning, or both         |
| Namespace filter | DynaKube spec                       | Limit to specific namespaces     |
| Event reasons    | Settings                            | Include/exclude specific reasons |

## Recommended Configuration

| Use Case                 | Configuration                       |
|--------------------------|-------------------------------------|
| <b>Full visibility</b>   | All event types, all namespaces     |
| <b>Warning focus</b>     | Warning events only, all namespaces |
| <b>Cost optimization</b> | Warning events, specific namespaces |

```
// Recent Kubernetes events
fetch logs
| filter matchesPhrase(log.source, "kubernetes") or matchesPhrase(log.source, "k8s")
| fields timestamp, content
| sort timestamp desc
| limit 50
```

```
// Warning events only
fetch logs
| filter matchesPhrase(content, "Warning")
| filter matchesPhrase(log.source, "kubernetes") or matchesPhrase(log.source, "k8s")
| fields timestamp, content
| sort timestamp desc
| limit 30
```

### 3. Container Log Collection

#### Log Collection Methods

| Method             | Source                  | Configuration            |
|--------------------|-------------------------|--------------------------|
| OneAgent           | Container stdout/stderr | Automatic with OneAgent  |
| Log Monitoring     | Mounted log files       | Custom paths in settings |
| Fluentd/Fluent Bit | External forwarder      | OTLP endpoint            |

#### OneAgent Log Collection

OneAgent automatically collects:

- Container stdout logs
- Container stderr logs
- Process-generated log files (with configuration)

#### Log Attributes

| Attribute                                       | Source              | Example                                                    |
|-------------------------------------------------|---------------------|------------------------------------------------------------|
| <code>k8s.namespace.name</code>                 | Container metadata  | <code>checkout</code>                                      |
| <code>k8s.pod.name</code>                       | Container metadata  | <code>checkout-api-abc123</code>                           |
| <code>k8s.container.name</code>                 | Container metadata  | <code>api</code>                                           |
| <code>dt.entity.container_group_instance</code> | Entity relationship | <code>CONTAINER_GROUP_INSTANCE-XXX</code>                  |
| <code>loglevel</code>                           | Parsed from content | <code>ERROR</code> , <code>WARN</code> , <code>INFO</code> |

## DynaKube Log Configuration

```
spec:
  oneAgent:
    cloudNativeFullStack:
      env:
        - name: ONEAGENT_ENABLE_LOG_ANALYTICS
          value: "true"
```

```
// Container logs by namespace
fetch logs
| filter isNotNull(k8s.namespace.name)
| summarize logCount = count(), by:{k8s.namespace.name}
| sort logCount desc
| limit 15
```

```
// Error logs with Kubernetes context
fetch logs
| filter loglevel == "ERROR" or loglevel == "SEVERE"
| filter isNotNull(k8s.namespace.name)
| fields timestamp, k8s.namespace.name, k8s.pod.name, content
| sort timestamp desc
| limit 30
```

## 4. OpenPipeline for K8s Logs

---

### Log Processing Pipeline

OpenPipeline processes logs before storage:

```
Log Ingest → Routing → Processing → Storage
              ↓           ↓
          Match rules  Transform, enrich
```

## Common Processing Rules

| Rule Type        | Use Case         | Example                  |
|------------------|------------------|--------------------------|
| <b>Parse</b>     | Extract fields   | JSON parsing, regex      |
| <b>Transform</b> | Modify content   | Rename fields, mask data |
| <b>Filter</b>    | Drop logs        | Remove debug logs        |
| <b>Route</b>     | Direct to bucket | By namespace or app      |

### Example: Parse JSON Logs

```
# OpenPipeline configuration
pipelines:
  - name: k8s-json-logs
    routes:
      - match: k8s.namespace.name exists
    processing:
      - type: json
        source: content
```

### Example: Filter Debug Logs

```
pipelines:
  - name: k8s-filter-debug
    routes:
      - match: k8s.namespace.name exists and loglevel == "DEBUG"
    processing:
      - type: drop
```

## 5. Event Analysis Patterns

---

### Key Event Reasons to Monitor

| Reason                  | Meaning                   | Action                         |
|-------------------------|---------------------------|--------------------------------|
| <b>FailedScheduling</b> | Pod can't be scheduled    | Check resource availability    |
| <b>FailedMount</b>      | Volume mount failed       | Check PV/PVC config            |
| <b>BackOff</b>          | Container restart backoff | Check logs, fix crash          |
| <b>Unhealthy</b>        | Probe failed              | Check probe config, app health |
| <b>Evicted</b>          | Pod evicted from node     | Check node pressure            |
| <b>OOMKilled</b>        | Out of memory             | Increase limits                |

```
// Failed scheduling events
fetch logs
| filter matchesPhrase(content, "FailedScheduling")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// Pod restart events (BackOff, CrashLoopBackOff)
fetch logs
| filter matchesPhrase(content, "BackOff") or matchesPhrase(content,
"CrashLoopBackOff")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// Volume mount failures
fetch logs
| filter matchesPhrase(content, "FailedMount") or matchesPhrase(content,
"FailedAttachVolume")
| fields timestamp, content
| sort timestamp desc
| limit 20
```



```
// Event frequency by reason (last 24h)
fetch logs, from: now() - 24h
| filter matchesPhrase(log.source, "kubernetes") or matchesPhrase(log.source, "k8s")
| filter matchesPhrase(content, "Warning")
| summarize eventCount = count(), by:{timeBucket = bin(timestamp, 1h)}
| sort timeBucket asc
```

## 6. Log Analysis Patterns

---

### Error Log Investigation

```
// Pattern: Find errors with full context
fetch logs
| filter loglevel == "ERROR"
| filter k8s.namespace.name == "checkout"
| fields timestamp, k8s.pod.name, content
| sort timestamp desc
| limit 50
```

### Log Volume Analysis

```
// Pattern: Identify noisy pods
fetch logs, from: now() - 1h
| summarize count = count(), by:{k8s.pod.name}
| sort count desc
| limit 10
```

### Exception Tracking

```
// Pattern: Find stack traces
fetch logs
| filter matchesPhrase(content, "Exception") or matchesPhrase(content, "Traceback")
| fields timestamp, k8s.namespace.name, content
| sort timestamp desc
```

```
// Log volume by pod (find noisy pods)
fetch logs, from: now() - 1h
| filter isNotNull(k8s.pod.name)
| summarize logCount = count(), by:{k8s.pod.name}
| sort logCount desc
| limit 15
```

```
// Exception and error messages
fetch logs
| filter matchesPhrase(content, "Exception") or matchesPhrase(content,
"error") or matchesPhrase(content, "failed")
| filter isNotNull(k8s.namespace.name)
| fields timestamp, k8s.namespace.name, k8s.pod.name, content
| sort timestamp desc
| limit 30
```

```
// Log level distribution by namespace
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name) and isNotNull(loglevel)
| summarize count = count(), by:{k8s.namespace.name, loglevel}
| sort count desc
| limit 30
```

## 7. Alerting on Events and Logs

---

### Event-Based Alerts

| Alert                    | Condition                             | Severity |
|--------------------------|---------------------------------------|----------|
| <b>Failed Scheduling</b> | FailedScheduling events > 5 in 10 min | Warning  |
| <b>Crash Loop</b>        | CrashLoopBackOff events               | Warning  |
| <b>OOM Kills</b>         | OOMKilled events                      | Critical |
| <b>Volume Failures</b>   | FailedMount events                    | Critical |

## Log-Based Alerts

| Alert           | Condition                  | Severity |
|-----------------|----------------------------|----------|
| Error Spike     | Error log count > baseline | Warning  |
| Critical Errors | Specific error patterns    | Critical |
| No Logs         | Log volume drops to 0      | Warning  |

## Custom Metric Events

Create metric events for log-based alerting:

1. Navigate to **Settings > Anomaly detection > Custom events**
2. Create event based on log count metric
3. Set thresholds and notification targets

## Next Steps

With event and log monitoring configured, proceed to:

| Next Notebook              | Topic                    |
|----------------------------|--------------------------|
| K8S-08: DQL for Kubernetes | Advanced query patterns  |
| K8S-09: Troubleshooting    | Debugging K8s monitoring |

## Summary

In this notebook, you learned:

- Kubernetes event types and sources
- Event ingestion configuration via DynaKube
- Container log collection methods
- OpenPipeline for log processing
- Event analysis patterns for common issues

- Log analysis patterns for debugging
  - Alerting strategies for events and logs
- 

## References

---

- [Kubernetes Log Monitoring](#)
  - [Kubernetes Events](#)
  - [OpenPipeline](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*